

Pipe, Redirect e Filtri

Su Linux ogni processo lavora con tre flussi di dati:

Flusso	Numero	Descrizione
stdin	0	Input standard — tastiera
stdout	1	Output standard — terminale
stderr	2	Output degli errori — terminale

Redirect e Pipe servono a reindirizzare questi flussi: prendere input da un file, scrivere output su un file, oppure passare l'output di un comando direttamente a un altro comando.

Per i test ho creato un file di esempio con `cat > frutta.txt << 'EOF'`, che accetta righe finché non incontra una riga contenente solo EOF (delimitatore di fine heredoc).

Output — Redirect

> — Sovrascrittura

```
echo "primo contenuto" > output.txt
```

L'operatore > prende lo stdout di un comando e lo scrive in un file, **sovrascrivendone sempre il contenuto**.

>> — Append

```
echo "secondo contenuto" >> output.txt
```

L'operatore >> prende lo stdout di un comando e lo scrive in un file **in coda al contenuto esistente**. Usato nei sistemi di logging per aggiungere eventi senza cancellare la storia precedente.

Input — Redirect

< — Input da file

```
wc -l < frutta.txt
```

L'operatore < fornisce il contenuto di un file come stdin a un comando.

Errori — Redirect

2> — Redirect stderr

```
cat file_che_non_esiste.txt 2> errori.txt
```

L'operatore 2> reindirizza i messaggi di errore dentro a un file.

2>&1 — Stderr verso stdout

```
cat frutta.txt file_inesistente.txt > tutto.txt 2>&1
```

Con 2>&1 indico di mandare **stderr** dove sta andando **stdout**. Così sia l'output normale che i messaggi di errore finiscono nello stesso file.

Pipe

Il carattere | connette lo **stdout** di un comando allo **stdin** del successivo:

```
comando1 | comando2 | comando3
```

Posso ad esempio ordinare le righe di un file alfabeticamente e rimuovere le ripetizioni:

```
cat frutta.txt | sort | uniq
```

Oppure contare quante righe diverse ci sono:

```
cat frutta.txt | sort | uniq | wc -l
```

Oppure stampare ogni riga unica con il numero di occorrenze:

```
cat frutta.txt | sort | uniq -c
```

Comandi utili

```
head -N file      # stampa le prime N righe
tail -N file      # stampa le ultime N righe
tail -f file      # segue il file in tempo reale (utile per monitorare log)
```

grep — ricerca nel testo

```
grep "testo" file  # stampa le righe che contengono "testo"
```

Flag utili:

Flag	Funzione
grep -c	Restituisce il numero totale di righe che contengono l'occorrenza (un solo numero)
grep -n	Stampa le righe corrispondenti con il loro numero di riga
grep -v	Stampa le righe che non contengono l'occorrenza
grep -i	Ricerca case-insensitive (ignora maiuscole/minuscole)

Esercizi — risposte

1. Quante righe contiene /etc/passwd?

```
cat /etc/passwd | wc -l # → 24
```

2. Quanti utenti distinti sono in /etc/passwd?

```
cat /etc/passwd | cut -d: -f1 | sort | uniq | wc -l # → 24
```

cut -d: -f1 estrae il primo campo usando : come delimitatore (il nome utente).
sort | uniq rimuove eventuali duplicati prima di contare.

3. Quali righe di /var/log/syslog contengono la parola “error” (maiuscolo o minuscolo)?

```
sudo grep -i "error" /var/log/syslog # → 2 risultati
```

Il flag -i gestisce entrambi i casi in un solo comando.

4. Ultime 5 righe di /var/log/auth.log

```
sudo tail -5 /var/log/auth.log
```